Universidad Laica Eloy Alfaro de Manabí

Facultad ciencias de la vida y tecnología

Carrera: ING. Software

Tema:

Nexocommerce



Autores:

Cardenas Ávila Emilio Sleimen

Michael Agustin Intriago Benitez

Anthony Axel Mejia Ordoñez

Melanie Dennise Perez Posligua

Nathalia Aracely Angulo Medranda

Docente:

Ing. Junior Zamora

Asignatura:

Integración e Implementación de Software

Periodo:

2026 - “A”

Fecha de entrega:

25/09/2026

1 tema del proyecto

Nombre provisional: NexoCommerce. Sistema de comercio electrónico para centralizar el catálogo, la personalización de productos, los pedidos y sus comprobantes de pago en Aesca, emprendimiento de repostería, detalles y sublimación de Montecristi.

La propuesta integra una aplicación web y una aplicación móvil que consultarán la misma API y la misma información. Su despliegue académico se realizará en cinco laptops conectadas a un switch, con responsabilidades separadas y una experiencia de uso unificada.

2 problemática a resolver

El contexto registrado en los requisitos del proyecto corresponde a Aesca, en Montecristi, un emprendimiento que comercializa repostería, detalles y artículos de sublimación mediante redes sociales y coordina manualmente sus pedidos por WhatsApp. La necesidad se ubica en la atención comercial y en la organización de la producción y las entregas.

Negocio beneficiario: Aesca.
Ubicación: Montecristi, Manabí, Ecuador.

En este proceso, el cliente consulta productos y acuerda por mensajes las características de su compra. La coordinación manual obliga a revisar conversaciones para recuperar diseños, cantidades, fechas y comprobantes. Los requisitos existentes identifican la necesidad de un catálogo, una agenda de pedidos y una verificación de pagos centralizados.

Problema central y dificultades

La información comercial queda dispersa entre publicaciones, conversaciones y archivos. Esto dificulta consultar una versión única del pedido y relacionar sus especificaciones con la fecha de entrega y el pago. También exige repetir respuestas sobre precios, opciones de personalización y avance del trabajo. Los errores, retrasos o pérdidas de información son riesgos a validar con el negocio; no se presentan como resultados medidos.

Usuarios y mejora esperada

Clientes: necesitan consultar opciones, definir su pedido y conocer su estado. **Administrador o propietario:** necesita mantener el catálogo, revisar pagos y organizar entregas. Los roles técnicos documentados son ADMIN y CLIENTE; no se define un tercer rol independiente.

Se espera disponer de un registro único por pedido, reducir la transcripción manual y facilitar el seguimiento desde web y móvil. El levantamiento deberá contrastar tiempos de atención, correcciones de pedidos y consultas de estado antes y después del uso del sistema. No se fija una reducción porcentual sin una medición inicial.

3 objetivos y alcance inicial

Objetivo general

Diseñar e implementar una solución distribuida que centralice la gestión comercial del emprendimiento y permita registrar, consultar y dar seguimiento a pedidos personalizados desde aplicaciones web y móvil conectadas a servicios compartidos.

Objetivos específicos

1. Organizar productos, categorías, precios, imágenes y opciones de personalización en un catálogo común.
2. Registrar pedidos con sus detalles, fecha solicitada, comprobantes y estados.
3. Proporcionar al administrador una agenda de pedidos y herramientas de validación de pagos.
4. Integrar los cinco nodos y demostrar continuidad de atención ante la caída de una instancia del backend.

Alcance funcional documentado

Incluye registro e inicio de sesión, catálogo de las tres líneas de producto, selección de diseños o plantillas, carrito, registro de pedidos, agenda administrativa, carga y revisión de comprobantes, consulta de estados y notificaciones dentro del sistema. Los datos de negocio serán compartidos; las funciones visibles dependerán del rol del usuario.

El alcance incluye pagos por pasarela y por transferencia con comprobante, autenticación mediante proveedor externo y verificación de correo. La pasarela y los servicios de identidad y correo requieren configuración, credenciales y conectividad

externa. Una carga de comprobante no confirmará automáticamente el pago; el administrador deberá revisarlo.

La configuración de la tienda y de la capacidad de producción forma parte de los módulos documentados. La solución busca ser reutilizable y parametrizable, con Aesca como negocio de aplicación. Los README no definen facturación electrónica, contabilidad integral ni seguimiento GPS como funcionalidades del proyecto.

Validación del contexto

El levantamiento se apoyará en una entrevista al responsable del emprendimiento y en la revisión autorizada de ejemplos anonimizados de pedidos. Se deberá confirmar cómo se calculan precios, qué fechas se aceptan, quién revisa los pagos y cuáles son los estados de producción y entrega. Los hallazgos se registrarán en una ficha con fecha, responsable y acuerdos.

Base documental

La descripción del proceso proviene de la especificación de requisitos conservada en el proyecto. El negocio beneficiario es Aesca, ubicado en Montecristi. La evidencia de entrevista deberá incorporarse para validar las reglas y dificultades del proceso. Esta propuesta define funcionalidades y despliegue previstos; no afirma que ya estén implementados o probados.

4 requerimientos funcionales iniciales

Los identificadores mantienen la correspondencia con los requisitos del proyecto. Cada resultado indica una comprobación inicial esperada.

ID	Funcionalidad y actor	Resultado verificable
RF-01 / 02	El visitante podrá registrarse y verificar su correo; el usuario podrá iniciar sesión con credenciales u OAuth y cerrar sesión.	Se rechazan correos duplicados y accesos inválidos; cada rol accede a sus funciones.
RF-03 / 04	El administrador podrá gestionar categorías y publicar novedades con imágenes.	Una publicación activa aparece en ambos clientes al actualizar la consulta.

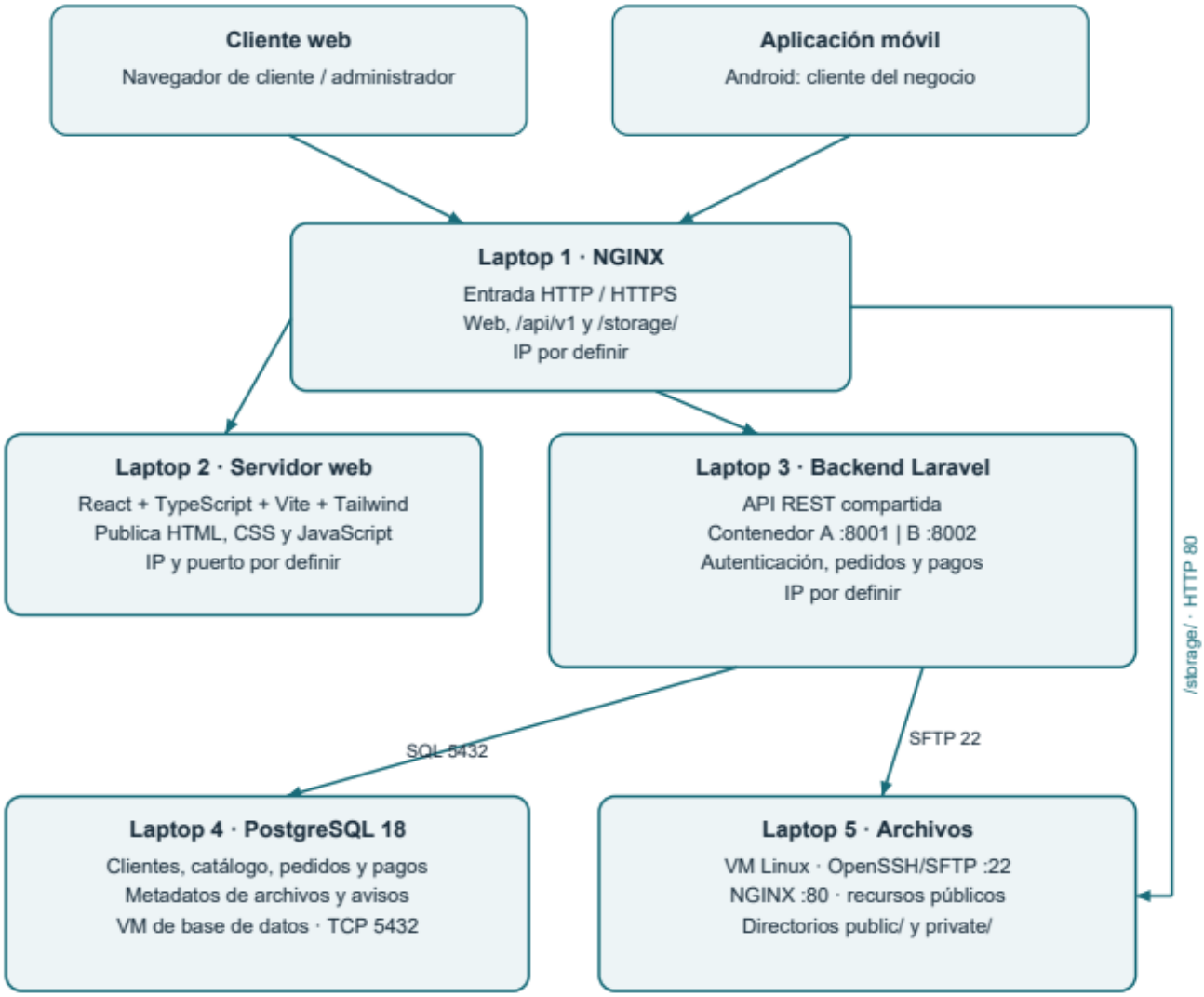
RF-05 / 06	El cliente podrá recibir recomendaciones de tortas por personas, elegir diseño o solicitar cotización de uno propio.	Se muestran opciones y adicionales; el diseño propio queda sujeto a cotización.
RF-07 / 08	El administrador podrá gestionar detalles; el cliente podrá consultar fichas y agregar cantidades al carrito.	Se conserva el precio y se impiden cantidades superiores al stock disponible.
RF-09 / 10 / 11	El administrador gestionará sublimables; el cliente podrá elegir plantilla o cargar diseño con vista previa referencial.	El diseño seleccionado o validado queda asociado al artículo del pedido.
RF-12	El cliente podrá agregar, modificar y retirar productos del carrito.	El sistema recalcula cantidades y totales y valida los datos antes de confirmar.
RF-13	El administrador podrá consultar la agenda, revisar pedidos y actualizar sus estados.	Cada pedido muestra fecha y estado; el cliente consulta el cambio desde su cuenta.
RF-14	El cliente podrá pagar por pasarela o adjuntar una transferencia que el administrador aprobará o rechazará.	Se registra el estado confirmado por el proveedor o la revisión manual, sin guardar datos de tarjeta.
RF-15	El cliente podrá consultar avisos sobre cambios de estado y de pago.	El aviso pertenece al usuario correcto y puede consultarse desde web y móvil.
RF-16	El administrador podrá cargar y administrar imágenes y plantillas; el sistema validará los archivos.	Se rechazan tipos o tamaños no permitidos; las referencias quedan asociadas a su recurso.

Reglas transversales

El cliente solo consultará sus pedidos y comprobantes. Los precios se validarán en el backend y se conservarán en el detalle del pedido. La agenda respetará la capacidad de producción. Pedido: PENDIENTE, EN_PREPARACION, LISTO, ENTREGADO. Pago: PENDIENTE, APROBADO, RECHAZADO. Los límites de archivo deberán unificarse entre los contratos y los clientes.

5 arquitectura propuesta

Las cinco laptops se conectarán por cable a un switch de la misma LAN. El navegador y la aplicación Android accederán al punto de entrada común. Para un teléfono físico se incorporará un punto de acceso Wi-Fi unido al switch; también podrá utilizarse un emulador con conectividad hacia la LAN.



6 responsabilidad de cada nodo

Nodo y responsable	Responsabilidad específica en NexoCommerce
Laptop 1 · Michael NGINX	Recibir accesos web, consultas de catálogo, pedidos y cargas de comprobantes. Enviar / al servidor web, /api/v1 a A o B y /storage/ al NGINX de archivos. Terminar HTTPS, registrar accesos y retirar temporalmente instancias con fallos detectados.
Laptop 2 · Nathalia Servidor web	Publicar los archivos de la aplicación React: catálogo, carrito, seguimiento y panel administrativo. El navegador ejecuta la interfaz y consulta la API común; este nodo no calcula los precios definitivos ni persiste pedidos.
Laptop 3 · Anthony Backend	Ejecutar al menos dos contenedores Laravel. Validar usuarios y permisos, calcular importes, registrar pedidos, revisar pagos y producir avisos. Coordinar el acceso a PostgreSQL y al servidor de archivos.
Laptop 4 · Melanie Base de datos	Persistir usuarios, categorías, productos, configuraciones, carrito, pedidos, detalles, pagos, estados y notificaciones. Guardar metadatos y referencias de archivos; aplicar integridad y respaldos.
Laptop 5 · Emilio Servidor de archivos	Almacenar imágenes del catálogo, diseños personalizados, plantillas y comprobantes. Transferir por SFTP con el backend y servir solo recursos públicos mediante NGINX. Conservar los binarios fuera de PostgreSQL y de los contenedores del backend.

Reglas de comunicación propuestas

Origen y destino	Acceso permitido
Clientes → Laptop 1	HTTP en ejemplos de desarrollo; HTTPS 443 previsto para acceso seguro.
Laptop 1 → Laptops 2 y 3	Puerto web por acordar; HTTP 8001/8002 al backend.
Laptop 3 → Laptop 4	TCP 5432 con usuario de base de datos de privilegios mínimos.
Laptop 3 → Laptop 5	SFTP sobre SSH 22/TCP, con cuenta y clave exclusivas.
Laptop 1 → Laptop 5	HTTP 80/TCP para /storage/ y recursos públicos.

Los clientes no accederán directamente a PostgreSQL ni a SFTP. Las imágenes públicas se consultarán mediante /storage/ en la entrada común. Los comprobantes y diseños privados se descargarán mediante la API con autorización. Laravel usará el adaptador SFTP de Flysystem; no se requiere crear una API adicional en la VM de archivos.

7 Rol de cada cliente

Aplicación web

El usuario con rol ADMIN utilizará el panel web para mantener categorías y productos, publicar novedades, revisar diseños y comprobantes, consultar la agenda y actualizar pedidos. Los compradores podrán acceder al catálogo, personalizar artículos, gestionar su carrito y consultar sus compras desde un navegador.

Aplicación móvil

La aplicación Android, desarrollada con Kotlin, Jetpack Compose y Material 3, estará dirigida principalmente al comprador. Permitirá registrarse, iniciar sesión, consultar productos, seleccionar opciones, adjuntar diseños o comprobantes y revisar pedidos y avisos. Emilio será responsable de su desarrollo, además del nodo de archivos. La aplicación se ejecutará en un dispositivo o emulador; no constituye un sexto servidor.

Información compartida y clientes adicionales

Web y móvil enviarán sus operaciones a /api/v1 mediante NGINX. Un pedido creado en el móvil aparecerá en el panel web al consultar la API; un cambio administrativo será visible al actualizar la consulta desde el móvil. No habrá bases independientes por cliente. No se incluyen clientes adicionales en el alcance inicial.

8 Flujo general de funcionamiento

Ejemplo de un pedido de sublimación con diseño propio

1. El comprador inicia sesión en Android, selecciona un artículo de sublimación e indica cantidad, fecha solicitada y diseño.
2. La aplicación carga el diseño por HTTPS al punto de entrada de la Laptop 1. NGINX dirige la solicitud a una instancia disponible del backend en la Laptop 3.
3. El backend verifica el token, el tipo y el tamaño del archivo. Envía el binario por SFTP a la carpeta privada de la Laptop 5 y registra su identificador y propietario en PostgreSQL, Laptop 4.

4. El comprador registra el pedido, pendiente de pago. El backend valida la referencia del diseño, calcula el total y guarda el pedido y sus detalles en una transacción. Devuelve el número del pedido a través de NGINX.
5. El administrador abre el panel web, cuya interfaz se descarga desde la Laptop 2 a través de NGINX. El navegador consulta la misma API y obtiene el pedido guardado.
6. El administrador revisa el pedido y actualiza su estado. El backend guarda el cambio y el aviso; el comprador los consulta desde Android.

Ruta principal: Usuario → App móvil → NGINX → Backend A o B → PostgreSQL → Backend → NGINX → App móvil.
Ruta del diseño: Backend → SFTP 22 → Servidor de archivos → Backend.

Si falla la carga del diseño, la API informará el error y no confirmará una asociación inexistente. El backend deberá evitar duplicados ante reintentos y registrar errores controlados. Un comprobante posterior seguirá el mismo recorrido de archivo, con su propio estado de revisión.

9 integración y validación previstas

El despliegue separará las variables de entorno del código, utilizará Git y contendrá al menos las dos instancias del backend en Docker. Los registros incluirán identificador de solicitud e instancia, sin contraseñas ni tokens. Se configurarán límites de carga, tiempos de espera y respuestas de error controladas. Se prepararán respaldos de la base de datos y de los archivos vinculados.

NGINX distribuirá las consultas y detectará fallos de instancia mediante comprobaciones de disponibilidad y la configuración de upstream. No se reintentará automáticamente una escritura que pueda duplicar una operación. La continuidad cubre la caída de un contenedor: si se apaga la Laptop 3, ambas instancias se detienen. Los demás nodos también son puntos únicos de fallo en esta arquitectura académica.

Prueba prevista	Evidencia y resultado esperado
Integración web y móvil	Crear un pedido desde Android, consultarlo y actualizarlo en web y verificar el mismo identificador y estado en móvil.
Balanceo y caída de instancia	Identificar respuestas atendidas por A y B en los logs. Detener A y comprobar que nuevas consultas se atienden por B tras la detección del fallo.
Servidor de archivos desconectado	Mostrar error controlado en cargas y descargas. Mantener disponibles las consultas textuales de pedidos y catálogo que no requieren archivos.
Base de datos detenida	Responder temporalmente con error de servicio sin confirmar pedidos no persistidos. Registrar el fallo y comprobar recuperación al restablecerla.
Permisos y consistencia	Intentar consultar un comprobante ajeno y verificar rechazo; comprobar que una carga fallida no deja una referencia de archivo inexistente.

Coherencia entre problema y solución

La dispersión de productos se atiende con el catálogo y las publicaciones (RF-03 a RF-11), visibles en ambos clientes. La coordinación manual se atiende con carrito, pedido y agenda (RF-12 y RF-13), procesados por el backend y persistidos en PostgreSQL. La separación entre comprobante y pedido se resuelve mediante la revisión de pagos y la gestión de archivos (RF-14 y RF-16). La falta de seguimiento se atiende con los estados y avisos compartidos (RF-15).

10 arquitectura lógica y servicios externos

Un backend modular compartido

La Laptop 3 ejecutará dos instancias del mismo monolito modular Laravel. Sus módulos son autenticación, usuarios, tienda, categorías, productos, publicaciones, producción, carrito, pedidos, pagos, notificaciones y multimedia. La organización prevista separa Dominio, Aplicación, Infraestructura y Http. No se trata de un servidor distinto por cada módulo.

El proceso SDD define contratos antes de implementar endpoints. Web y móvil consumirán /api/v1, con respuestas JSON comunes y tokens Bearer de Sanctum. Las

dos instancias accederán a la misma base de datos y al mismo almacenamiento remoto. La configuración de tienda y producción se administrará desde web mediante esa API.

Servicios externos contemplados

Servicio	**Conexión e intervención prevista**
Identidad OAuth	Cliente → proveedor de identidad → flujo de autenticación del backend. Google figura en requisitos; el contrato API enumera GOOGLE y FACEBOOK. La habilitación de proveedores requiere configuración.
Pasarela de pagos	Cliente → pasarela para introducir datos de tarjeta; backend ↔ pasarela para crear o verificar operaciones. Las confirmaciones entrantes deberán pasar por NGINX hacia la API. Proveedor y contrato por concretar.
Correo electrónico	Backend → servicio de correo para verificación de cuenta y avisos cuando corresponda. Las notificaciones dentro de la plataforma se conservan en PostgreSQL. Proveedor y transporte por definir.

Estos servicios complementan los cinco nodos obligatorios; no los sustituyen. Necesitan acceso a Internet. Si la pasarela utiliza callbacks, una LAN aislada no será accesible desde el proveedor: el método de prueba y la conectividad deberán acordarse en el despliegue.

Lectura de archivos públicos y privados

Imagen del catálogo: Web o móvil → NGINX Laptop 1 /storage/ → NGINX Laptop 5 → archivo público → cliente por la misma entrada.
Comprobante privado: Web o móvil → NGINX Laptop 1 → backend → validación de permisos → SFTP Laptop 5 → backend → cliente. Nunca se publica el directorio privado.

La VM de archivos propuesta utiliza Ubuntu Server LTS, red puente sobre Ethernet, almacenamiento persistente, OpenSSH y NGINX. Se proponen 2 CPU virtuales, 2 a 4 GB de RAM y 30 a 40 GB de disco; son valores de planificación, no exigencias de las bases.